Pathfinder

A Comparative Shortest-Path Platform on DC Road Data

1.88 M graph nodes **3.96 M** directed edges **138 540** indexed POIs

Data Structures & Algorithms — Project Defense

Problem & Dataset

Directed, weighted graph with four cost objectives

Four cost models on the same graph

- distance — metres
- time — metres / per-class speed
- balanced — convex blend
- safe_walk — time × pedestrian safety

User actions → DS&A problems

- click map → nearest node (2-D spatial lookup)
- keyword → POI list (ranked string search)
- pick A, B → shortest path (graph algorithm)
- compare algorithms (parallel execution)

Dataset OpenStreetMap DC area • 900 MB GeoJSON source • loaded once into memory, served from binary cache on subsequent starts

System Overview

Three tiers, no database on the critical path

Client

React + Leaflet. Renders map, pins, routes. Talks JSON over REST.

Service

Spring Boot. RoutingService · SearchService · NearestService · AnnotationService.

Memory

In-memory graph · kd-tree · inverted index — built once from binary cache.

Disk

GeoJSON (source) + binary cache (graph + string table) — read-only on startup.

Graph · Adjacency List + Lazy Reverse

Every routing request iterates outgoing edges

- Structure: Map<String, List<Edge>>
- Edge carries distance, highway, speed, oneway, road name
- Matrix would need $\Theta(V^2) \approx 28$ TB at 1 bit per cell
- Adjacency list uses $\Theta(V + E) \approx 6$ M cells
- Reverse adjacency is built lazily on first bidirectional call
- Double-checked volatile idiom — unidirectional queries pay nothing

Operation	Time
addNode / addEdge	O(1) amortised
getOutgoing(v)	O(deg v)
getNode(id)	O(1)
getIncoming(v) — first	O(V + E) build
getIncoming(v) — after	O(deg v)

Graph.java · Edge.java · ReverseEdge.java

Dijkstra · Baseline Routing

Optimal on non-negative weights

- Complexity: O((V + E) log V) with a binary heap
- Invariant: when v is popped, dist[v] is final
- Proof: remaining nodes have dist ≥ dist[v]; edges cannot decrease
- Uses the same PathCostModel as A* and bi-Dijkstra

Non-negativity of every cost model

- distance — metric, ≥ 0
- time — metres / speed with speed > 0
- balanced — convex combination of two ≥ 0 terms
- safe_walk — time × positive safety factor

DijkstraShortestPath.java · PathCostModel.java · RoutingCostPolicy.java

Bidirectional Dijkstra

Two frontiers, meet in the middle

- Alternate forward from s and backward from t
- μ = best s–t cost via any node both sides have touched
- Stop when top(openF) + top(openB) ≥ μ
- Correctness: any better path's meeting node would still sit in one frontier
- Needs symmetric, non-negative edge costs (all four objectives satisfy)

UNREACHABLE PAIR

Dijkstra 2 245 ms

Bi-Dijkstra

0.17 ms

≈ 12 000× faster

on disconnected components

BidirectionalDijkstra.java · Graph.getIncoming()

A* · Haversine Heuristic

Admissible — never overestimates remaining cost

- $f(v) = g(v) + h(v)$, $h(v)$ = great-circle to target, scaled per objective
- Proof sketch: Σ segment haversines $\geq H(v, t)$ by triangle inequality
- Scaling by $1 / S_max$ or MIN_SAFETY preserves the lower bound
- Same $O((V + E) \log V)$ worst case, far fewer expansions in practice

Objective	Admissible	Condition
distance	✓	triangle inequality on sphere
time	✓	heuristic uses S_max
balanced	✓	S_ref ≤ S_max enforced in config
safe_walk	✓	multiplied by MIN_SAFETY lower bound
+ preferMainRoad=true	⚠	multiplier < 1 — known caveat

AStarShortestPath.java · admissibility.md

Priority Queue · Binary Heap

Open set for every shortest-path algorithm

- JDK PriorityQueue — binary min-heap
- Used by Dijkstra, A*, and both halves of bi-Dijkstra
- Lazy deletion: push fresh (v, betterG), skip stale entries on pop
- Avoids explicit decrease-key, keeps the heap a primitive

	Insert	Extract-min	Decrease-key
Binary heap	O(log V)	O(log V)	O(log V)
Fibonacci	O(1) amort.	O(log V) amort.	O(1) amort.
d-ary (d=4)	$O(\log_4 V)$	$O(d \cdot \log_4 V)$	$O(\log_4 V)$

Fibonacci's asymptotic edge is erased by pointer overhead and cascading cuts. Binary heap wins empirically.

Binary Graph Cache

Startup speedup from string-table de-duplication

- Problem: parsing 900 MB of GeoJSON costs ~10 s per restart
- Custom binary format — header · string table · nodes · adjacency · search items
- Shared string table de-duplicates highly repeated OSM tags
- Cache validity keyed on source (size, mtime) — not a slow content hash
- Atomic write — .tmp + Files.move with ATOMIC_MOVE fallback

cold	warm
13.8 s	**1.8 s**
parse + write	read cache

7.5× faster startup

Payload: ~1.5 GB naive UTF → **279 MB** with string table (1.96 M unique strings)

GraphCacheStore.java · GeoJsonLoader.java

Inverted Index + Levenshtein Fuzzy Match

Keyword search over 138 K points of interest

- Index: Map<token, List<itemIdx>> built once, linear pass
- Tokens: lowercased, accent-stripped, split on whitespace & punctuation
- Candidates: union of posting lists, plus prefix & fuzzy scan
- Levenshtein DP on two rolling rows, early-exit when row-min > max
- Length diff > max-edit-distance → skip without DP

Match type	Score
Exact	10
Prefix	5
Contains	2
Fuzzy (Levenshtein ≤ 2)	3
Token in name	*× 2 multiplier*

SearchService.java — buildTokenIndex · scoreItem · editDistance

2-D kd-tree · Nearest Node on Map Click

Fix for 1.88 M-node linear scan on every click

- Balanced kd-tree on (lat, lon · cos meanLat)
- cos(meanLat) projects lon so Euclidean ordering matches great-circle
- Flat int[] left / right / index arrays (~22 MB vs ~75 MB for objects)
- Build: in-place quickselect — O(N log N) expected
- Random pivot to avoid $O(N^2)$ on pre-sorted input
- Query: descend near, prune far when $diff^2 \geq bestSq$

LINEAR SCAN

87.6 ms

kd-TREE

40 µs

2 205× faster

NodeKdTree.java · NearestNodeService.java

Complexity & Empirical Results

50 random DC pairs · distance objective

Per-operation complexity

Operation	Time
Adjacency iteration	O(deg v)
Dijkstra / A* / Bi-Dij	O((V+E) log V)
Nearest node (kd-tree)	O(log V) exp.
kd-tree build	O(N log N) exp.
Keyword search	O(cand · |t|)
Graph cache load	O(V + E)

Routing benchmark (avg over 48 reachable pairs)

Algorithm	Relax.	Time
Dijkstra	2 164 413	1150 ms
A*	538 737	334 ms
Bidirectional	1 487 568	962 ms

A* visits 4× fewer edges than Dijkstra, 3.4× faster in wall time. **Bi-Dijkstra** gains modestly on reachable pairs — its killer case is **unreachable pairs (12 000× faster)**. All three algorithms returned identical distances on the 48 reachable pairs.

Raw CSV: analysis/results/benchmark-latest.csv

Live Demo Plan

About three minutes — data structures in action

1

Startup comparison *~30 s*

Delete cache → cold parse + write. Restart → warm load in ≈ 2 s.

2

Keyword search *~30 s*

Type 'pho 75', 'neu arlington', 'clarendn' (typo) — fuzzy match recovers it.

3

Nearest-node snap *~10 s*

Click random point on map → kd-tree returns nearest node + snap distance.

4

Routing comparison *~60 s*

Pick A, B → switch Dijkstra / A* / Bi-Dijkstra; then pick an unreachable pair.

5

Objective overlay *~30 s*

Enable compare mode — four coloured polylines for distance / time / balanced / safe_walk.

Takeaways

01

Adjacency list + binary heap + A* is the empirical winner

Haversine is a near-tight lower bound on DC road geometry — 4× fewer relaxations than Dijkstra.

02

Bidirectional Dijkstra's killer case is unreachability

~12 000× faster on disconnected pairs; modest gains on reachable pairs.

03

kd-tree is a 2 200× speedup for one UX-critical hot path

Flat int[] layout + cos-lat projection + quickselect build. Under 600 ms on 1.88 M nodes.

04

Every correctness claim has a written proof

Non-negativity, admissibility, bi-Dijkstra termination — each argued in code comments and analysis/.